

LAPORAN TUGAS

STURKTUR DATA DAN ALGORITMA

PENJELASAN TAMBAHAN TUGAS 5 HEAP & HEAPSORT



Dosen Pengampu:

Prof. Dr. Taufik Fuadi Abidin, M.Tech.

Dr. Ir. Irvanizam, S.Si., M.Sc., IPM., ASEAN Eng.

Disusun oleh:

Muhammad Aqil Mubarak (250810701100003)

JURUSAN INFORMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS SYIAH KUALA

BANDA ACEH

2026

PROBLEM TUGAS 5

Problem 1

Anda dimintakan untuk membuat sebuah program yang dapat mengurutkan bilangan desimal menggunakan metode heapsort. Program menerima input berupa n sebuah bilangan dimana $1 < n \leq 3000000$ dan kemudian mengacak n buah bilangan desimal (bilangan real antara 0 and 1). Program yang dibuat harus menyediakan MENU dan harus melakukan hal-hal sebagai berikut (sebagai contoh):

Masukkan Jumlah Bilangan (n): 200000

Random bilangan real dalam array selesai...

Pilihan:

- 1) Mengurutkan data bilangan real dengan metode HEAP SORT dan menyimpan hasil dalam file teks serta menampilkan waktu pengurutan
- 2) Tentukan jumlah bilangan dan acak ulang bilangan real dalam array
- 3) Selesai Pilihan anda:

Gunakan fungsi rand() untuk mengacak bilangan real tersebut. Informasi rinci tentang rand() dapat diperoleh di Internet, salah satunya pada URL berikut.
<https://www.naukri.com/code360/library/random-function-in-c>

Selanjutnya, program akan menampilkan menu dengan beberapa pilihan. Bila pilihan (1) dipilih, maka program akan mengurutkan data menggunakan metode heapsort dan menampilkan data yang telah besar ke kecil dan menyimpan hasil pengurutan dalam file teks, dan menampilkan jumlah waktu yang dibutuhkan untuk mengurutkan data tersebut dalam satuan detik atau milidetik.

Bila pilihan (2) yang dipilih, maka program akan meminta input nilai n kembali dan mengacak kembali n bilangan real ke dalam array. Program akan berhenti bila Pilihan (3) dipilih.

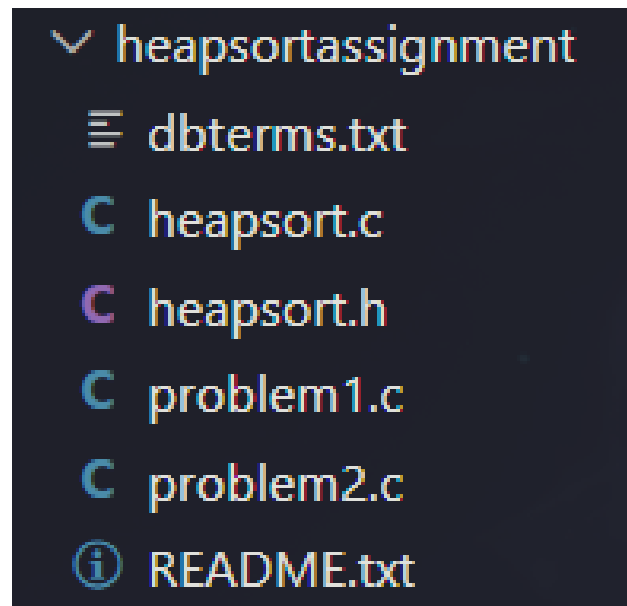
Problem 2

Anda dimintakan untuk membuat sebuah program yang dapat mengurutkan data berupa kata kata dari file teks menggunakan metode heap sort. Program menerima input file teks dari URL berikut sebagai contoh <https://informatika.usk.ac.id/tfa/ds/dbterms.txt>. Kata-kata dalam

file teks tersebut berjumlah 25988 (ada duplikasi) dan disusun secara tidak teratur. Program harus menyediakan MENU seperti pada soal 1, namun tanpa ada fitur pengacakan bilangan.

PENJELASAN TAMBAHAN

Dalam Pengerjaan solusi problem ini, saya menggunakan pendekatan modularitas, dimana program dipecah menjadi beberapa file terpisah yaitu file header dan file implementasi. Cara ini memisahkan logika dasar struktur data dengan logika utama penyelesaian pada problem, sehingga kode menjadi rapi dan mudah di kelola.



Gambar 1. Direktori File

Program dapat di compile dengan command:

`'gcc -Wall -pedantic -g -o problem1 problem1.c heapsort.c "`

dan

`'gcc -Wall -pedantic -g -o problem2 problem2.c heapsort.c "`

Problem 1

Pada Problem 1, tujuan utamanya adalah mengurutkan data bilangan real secara menurun atau *Descending* (dari nilai terbesar ke terkecil). Untuk mencapai ini dengan efisien menggunakan algoritma *Heapsort*, struktur awal yang dibangun haruslah Min-Heap. Pada *Min-Heap*, elemen induk (*root*) selalu memiliki nilai yang lebih kecil dibandingkan dengan anak-anaknya. Saat *root* terkecil ini diekstrak berulang kali ke bagian akhir *array*, elemen-elemen tersebut akan tersusun menjadi urutan yang menurun.

```

1 void minHeapify(double arr[], int n, int i, long long *swapCount) {
2     // Inisialisasi node terkecil sebagai root
3     int smallest = i;
4     int left = 2 * i + 1;
5     int right = 2 * i + 2;
6
7     // Cek apakah anak kiri lebih kecil dari root
8     if (left < n && arr[left] < arr[smallest]) {
9         smallest = left;
10    }
11
12    // Cek apakah anak kanan lebih kecil dari node terkecil saat ini
13    if (right < n && arr[right] < arr[smallest]) {
14        smallest = right;
15    }
16
17    // Jika node terkecil bukan root, lakukan pertukaran dan lanjut heapify
18    if (smallest != i) {
19        double temp = arr[i];
20        arr[i] = arr[smallest];
21        arr[smallest] = temp;
22        (*swapCount)++;
23
24        minHeapify (arr, n, smallest, swapCount);
25    }
26 }
27

```

Gambar 2. Fungsi minHeapify

Fungsi rekursif ini membandingkan *node* indeks ke-*i* dengan anak kiri (*left*) dan anak kanan (*right*). Jika ditemukan anak yang nilainya lebih kecil dari *smallest* saat ini, indeks *smallest* akan diperbarui. Jika *smallest* berubah posisi, dilakukan operasi pertukaran nilai (*temp*). Variabel *pointer* (**swapCount*)++ digunakan untuk merekam jumlah pertukaran yang terjadi. Fungsi ini kemudian memanggil dirinya sendiri untuk memastikan struktur *subtree* di bawahnya tetap memenuhi syarat *Min-Heap*.

Terakhir, program harus mampu menerima *input* jumlah bilangan desimal *n* yang ukurannya bisa menyentuh angka 3.000.000. Mengalokasikan *array* secara statis (seperti `double arr[3000000];`) di dalam fungsi *main* akan berakibat fatal berupa *Stack Overflow*.

```

1 if (arr != NULL) free(arr);

```

Gambar 3. Free memory

Output Problem:

```
o> .\problem1
Masukkan Jumlah Bilangan (n): 200000
Random bilangan real dalam array selesai...

Pilihan:
1) Mengurutkan data bilangan real dengan metode HEAP SORT dan menyimpan hasil dalam file teks serta menampilkan waktu pengurutan
2) Tentukan jumlah bilangan dan acak ulang bilangan real dalam array
3) Selesai
Pilihan anda: 1

Data berhasil diurutkan secara Descending dan disimpan di 'hasilsortingproblem1.txt'.
Waktu pengurutan : 0.057000 detik
Jumlah pertukaran : 3349625 kali

Pilihan:
1) Mengurutkan data bilangan real dengan metode HEAP SORT dan menyimpan hasil dalam file teks serta menampilkan waktu pengurutan
2) Tentukan jumlah bilangan dan acak ulang bilangan real dalam array
3) Selesai
Pilihan anda: █
```

Gambar 4. Output Pilihan 1

```
Pilihan:
1) Mengurutkan data bilangan real dengan metode HEAP SORT dan menyimpan hasil dalam file teks serta menampilkan waktu pengurutan
2) Tentukan jumlah bilangan dan acak ulang bilangan real dalam array
3) Selesai
Pilihan anda: 2
Masukkan Jumlah Bilangan (n) baru: 2000000
Random bilangan real dalam array selesai...

Pilihan:
1) Mengurutkan data bilangan real dengan metode HEAP SORT dan menyimpan hasil dalam file teks serta menampilkan waktu pengurutan
2) Tentukan jumlah bilangan dan acak ulang bilangan real dalam array
3) Selesai
Pilihan anda: █
```

Gambar 5. Output Pilihan 2

Memory Check Problem 1

```
eruma@kali: ~/Downloads/SDA-main/heapsortassignment
Session Actions Edit View Help
(eruma@kali)-[~/Downloads/SDA-main/heapsortassignment]
$ gcc -Wall -g -pedantic -o problem1 problem1.c heapsort.c
(eruma@kali)-[~/Downloads/SDA-main/heapsortassignment]
$ gcc -Wall -g -pedantic -o problem2 problem1.c heapsort.c
(eruma@kali)-[~/Downloads/SDA-main/heapsortassignment]
$ valgrind ./problem1
==3982== Memcheck, a memory error detector
==3982== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3982== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==3982== Command: ./problem1
==3982==
Masukkan Jumlah Bilangan (n): 2000000
Random bilangan real dalam array selesai...

Pilihan:
1) Mengurutkan data bilangan real dengan metode HEAP SORT dan menyimpan hasil dalam file
teks serta menampilkan waktu pengurutan
2) Tentukan jumlah bilangan dan acak ulang bilangan real dalam array
3) Selesai
Pilihan anda: 3
Program Selesai.
==3982==
==3982== HEAP SUMMARY:
==3982==   in use at exit: 0 bytes in 0 blocks
==3982==   total heap usage: 3 allocs, 3 frees, 16,002,048 bytes allocated
==3982==
==3982== All heap blocks were freed -- no leaks are possible
==3982==
==3982== For lists of detected and suppressed errors, rerun with: -s
==3982== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
(eruma@kali)-[~/Downloads/SDA-main/heapsortassignment]
$ █
```

Gambar 6. Memory Check Problem 1

Problem 2

Pada problem 2, data berupa kumpulan kata (berjumlah 25.988) harus diurutkan secara alfabetis dari A ke Z (*Ascending*). Oleh karena itu, arsitektur yang digunakan adalah Max-Heap. Karena data yang diproses adalah *array of strings* 2D, kita harus menggunakan fungsi bawaan `strcmp` (String Compare) dan `strcpy` (String Copy) dari *library* `<String.h>`.

```
1 void minHeapify(double arr[], int n, int i, long long *swapCount) {
2     // Inisialisasi node terkecil sebagai root
3     int smallest = i;
4     int left = 2 * i + 1;
5     int right = 2 * i + 2;
6
7     // Cek apakah anak kiri lebih kecil dari root
8     if (left < n && arr[left] < arr[smallest]) {
9         smallest = left;
10    }
11
12    // Cek apakah anak kanan lebih kecil dari node terkecil saat ini
13    if (right < n && arr[right] < arr[smallest]) {
14        smallest = right;
15    }
16
17    // Jika node terkecil bukan root, lakukan pertukaran dan lanjut heapify
18    if (smallest != i) {
19        double temp = arr[i];
20        arr[i] = arr[smallest];
21        arr[smallest] = temp;
22        (*swapCount)++;
23
24        minHeapify (arr, n, smallest, swapCount);
25    }
26 }
27
```

Gambar 7. Fungsi minHeapify

Fungsi `strcmp(arr[left], arr[largest]) > 0` bekerja mengevaluasi bobot alfabetis kata. `strcmp` akan mengembalikan nilai lebih dari 0 apabila *string* pertama secara alfabetis lebih besar dari *string* kedua (contoh: huruf "B" dibandingkan "A"). Ketika posisi perlu ditukar, penyalinan nilai tidak bisa menggunakan tanda `=` biasa, melainkan harus menggunakan `strcpy` untuk menyalin *array* karakter secara utuh ke dalam variabel penyangga sementara (`temp`).

Output Problem:

```
> .\problem2
Berhasil memuat 25988 kata dari dbterms.txt...

Pilihan:
1) Mengurutkan kata dengan metode HEAP SORT, menyimpan ke teks, dan menampilkan waktu
2) Selesai
Pilihan anda: 1

Data berhasil diurutkan dan disimpan di 'hasilsortingproblem2.txt'.
Waktu pengurutan : 14.00 milidetik
Jumlah pertukaran : 358896 kali

Pilihan:
1) Mengurutkan kata dengan metode HEAP SORT, menyimpan ke teks, dan menampilkan waktu
2) Selesai
Pilihan anda: █
```

Gambar 8. Output Pilihan 1

Memory Check Problem 2

```
(eruma@kali)-[~/Downloads/SDA-main/heapsortassignment]
$ valgrind ./problem2
==4532== Memcheck, a memory error detector
==4532== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==4532== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==4532== Command: ./problem2
==4532==
Berhasil memuat 25988 kata dari dbterms.txt...

Pilihan:
1) Mengurutkan kata dengan metode HEAP SORT, menyimpan ke teks, dan menampilkan waktu
2) Selesai
Pilihan anda: 2
Program Selesai.
==4532==
==4532== HEAP SUMMARY:
==4532==   in use at exit: 0 bytes in 0 blocks
==4532==   total heap usage: 4 allocs, 4 frees, 6,616 bytes allocated
==4532==
==4532== All heap blocks were freed -- no leaks are possible
==4532==
==4532== For lists of detected and suppressed errors, rerun with: -s
==4532== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

(eruma@kali)-[~/Downloads/SDA-main/heapsortassignment]
$ █
```

Gambar . Memory Check Problem 2

KESIMPULAN

Berdasarkan implementasi dan hasil uji coba pada Tugas 5, algoritma Heapsort terbukti sangat efisien dan tangguh dalam mengurutkan data berskala masif, seperti 3.000.000 elemen bilangan real maupun 25.988 baris kata, dalam waktu eksekusi yang sangat singkat. Keberhasilan ini didukung oleh fleksibilitas pemilihan arsitektur *Heap* yang tepat, yakni penggunaan Min-Heap untuk menghasilkan urutan menurun (*Descending*) pada bilangan desimal (Problem 1) dan Max-Heap yang dipadukan dengan manipulasi *string* (`strcmp` dan `strcpy`) untuk urutan alfabetis menaik (*Ascending*) pada kata (Problem 2). Selain itu, penerapan alokasi memori dinamis (`malloc`) terbukti menjadi langkah krusial untuk mencegah terjadinya *Stack Overflow* saat menangani alokasi jutaan data secara bersamaan. Secara keseluruhan, program ini telah berhasil memenuhi seluruh spesifikasi tugas dengan mengeksport hasil pengurutan secara aman ke dalam *file* teks (.txt) demi menjaga stabilitas memori, sembari tetap menampilkan metrik analitik berupa durasi waktu dan total pertukaran (*swap count*) secara akurat di layar *console* terminal.